Лабораторная работа № 6

Тема работы: Исследование особенностей разработки и анализ программного кода Java.

Цель работы: Исследовать особенности разработки и провести анализ программного кода Java.

Задание 1. Изучите графические возможности языка программирования Java и нарисуйте дом, используя различные геометрические фигуры и линии.

Задание 2. Выполните задание, приведенное в приложении А.

Оснащение работы: Методические указания к выполнению, ОС Windows, установщик JDK и Eclipse.

Основные теоретические сведения

Графический пользовательский интерфейс (GUI) – основной способ взаимодействия конечных пользователей с java-приложением. Для разработки прикладного программного обеспечения на языке Java, графического интерфейса приложений, обычно используются пакеты AWT и Swing.

– AWT (для доступа загружается пакет java.awt) содержит набор классов, позволяющих выполнять графические операции и создавать оконные элементы управления, подобно тому, как это делается в VBA и Delphi;

– Swing (для доступа загружается пакет javax.swing) содержит новые классы, в основном аналогичные AWT. К именам классов добавляется J (JButton, JLabel и др).

На данный момент основные классы для построения визуальных интерфейсов содержатся в пакете Swing. Из пакета AWT используются классы для обработки сообщений. Простейшее графическое приложение приведено ниже.

Технология Swing предоставляет механизмы для управления следующими аспектами представления:

клавиатура (Swing предоставляет способ перехвата пользовательского ввода);

цвета (Swing предоставляет способ менять цвета, которые вы видите на экране);

текстовое поле для ввода (Swing предоставляет текстовые компоненты для обработки всех повседневных задач).

JComponent

Базовым классом всей библиотеки визуальных компонентов Swing является JComponent. Это суперкласс других визуальных компонентов. Он является абстрактным классом, поэтому в действительности вы не можете создать JComponent, но он содержит сотни функций, которые каждый компонент Swing может использовать как результат иерархии классов. Класс JComponent обеспечивает инфраструктуру окрашивания для всех компонентов, он знает, как обрабатывать все нажатия клавиш на клавиатуре, его подклассы, следовательно, должны только прослушивать определенные клавиши. Класс JComponent также содержит метод add(), который позволяет

добавить другие объекты класса JComponent, так можно добавить любой Swing-компонент к любому другому для создания вложенных компонентов (например, JPanel, содержащую JButton, или даже более причудливые комбинации, например JMenu, содержащее JButton).

JLabel

Самым простым и в то же время основным визуальным компонентом в библиотеке Swing является JLabel, или «метка». К методам этого класса относится установка текста, изображения, выравнивания и других компонентов, которые описывает метка:

get/setText() – получить/установить текст в метке;

get/setIcon() – получить/установить изображение в метке;

get/setHorizontalAlignment – получить/установить горизонтальную позицию текста;

get/setVerticalAlignment() – получить/установить вертикальную позицию текста;

get/setDisplayedMnemonic() – получить/установить мнемонику (подчеркнутый символ) для метки;

get/setLabelFor() – получить/установить компонент, к которому присоединена данная метка; когда пользователь нажимает комбинацию клавиш Alt + мнемоника, фокус перемещается на указанный компонент.

JButton

Основным активным компонентом в Swing является Jbutton.

Методы, используемые для изменения свойств JButton, аналогичны методам JLabel (вы обнаружите, что они аналогичны для большинства Swing-компонентов). Они управляют текстом, изображениями и ориентацией:

get/setText() – получить/установить текст в кнопке;

get/setIcon() – получить/установить изображение в кнопке;

get/setHorizontalAlignment() – получить/установить горизонтальную позицию текста;

get/setVerticalAlignment() – получить/установить вертикальную позицию текста;

get/setDisplayedMnenomic() – получить/установить мнемонику (подчеркнутый символ), которая в комбинации с кнопкой Alt вызывает нажатие кнопки.

JFrame

КлассJFrame является контейнером, позволяющим добавлять к себе другие компоненты для их организации и предоставления пользователю.

JFrame выступает в качестве моста между независимыми от конкретной операционной системы Swing-частями и реальной операционной системой, на которой они работают. JFrame регистрируется как окно и таким образом получает многие свойств окна операционной системы: минимизация/максимизация, изменение размеров и перемещение. Хотя выполнении лабораторной работы достаточно считать JFrame палитрой, на

которой вы размещаете компоненты. Перечислим некоторые из методов, которые вы можете вызвать в JFrame для изменения его свойств:

get/setTitle() – получить/установить заголовок фрейма;

get/setState() – получить/установить состояние фрейма (минимизировать, максимизировать и т.д.);

is/setVisible() – получить/установить видимость фрейма, другими словами, отображение на экране;

get/setLocation() – получить/установить месторасположение в окне, где фрейм должен появиться;

get/setSize() – получить/установить размер фрейма;

add() – добавить компоненты к фрейму.

Схемы, модели и события

При построении визуальных приложений в Java нельзя просто случайно разместить их на экране и ожидать от них немедленной работы. Компоненты необходимо разместить в определенные места, реагировать на взаимодействие с ними, обновлять их на основе этого взаимодействия и заполнять данными. Для эффективной работы с визуальными компонентами необходима установка следующих трех архитектурных составляющих Swing.

1. Схемы (layout). Swing содержит множество схем, которые представляют собой классы, управляющие размещением компонентов в приложении и тем, что должно произойти с ними при изменении размеров окна приложения или при удалении или добавлении компонентов.

2. События (event). Программа должна реагировать на нажатия клавиш, нажатия кнопки мыши и на все остальное, что пользователь может сделать.

3. Модели (model). Для более продвинутых компонентов (списки, таблицы, деревья) и даже для некоторых более простых, например, JComboBox, модели – это самый эффективный способ работы с данными. Они удаляют большую часть работы по обработке данных из самого компонента (вспомните MVC) и предоставляют оболочку для общих объектных классов данных (например, Vector и ArrayList).

Особое внимание, в связи с необходимостью изображения динамических сцен на визуальных компонентах необходимо уделить классу Graphics 2D.

Рисование линий

Линии рисуются методом drawLine() формата:

```
void drawLine (int startX, int startY, int endX, int endY)
```

Данный метод отображает линию (в текущем цвете рисования), которая начинается в координатах startX, startY и заканчивается в endX, endY. Пример использования метода:

```
public void paint(Graphics g) {
 g.drawLine(0, 0, 100, 100);
}
```

Рисование прямоугольников

Методы drawRect() и fillRect() отображают соответственно рисованный и заполненный прямоугольник. Их форматы:

```
void drawRect(int top, int left, int width, int height)
void fillRect(int top, int left, int width, int height)
```

Координаты левого верхнего угла прямоугольника задаются в параметрах top и left, width и height, указывающих размеры прямоугольника (в пикселах).

```
public void paint(Graphics g)
{ g.drawRect(10, 10, 60, 50);
 g.fillRect(100, 10, 60, 50);
}
```

Рисование эллипсов и кругов

Для рисования эллипса используется drawOval(), а для его заполнения – fillOval(). Эти методы имеют форматы:

```
void drawOval(int top, int left, int width, int height)
void fillOval(int top, int left, int width, int height)
```

Пример

```
public void paint(Graphics g)
{ g.drawOval(10, 10, 50, 50);
 g.fillOval(100, 10, 75, 50);
}
```

Рисование дуг

Дуги можно рисовать методами drawArc() и fillArct(), используя форматы:

```
void drawArc(int top, int left, int width, int height, int начало, int конец)
void fillArc(int top, int left, int width, int height, int начало, int конец)
```

Дуга ограничена прямоугольником: левый верхний угол прямоугольника определяется параметрами top, left, а ширина и высота – параметрами width и height. Дуга рисуется от начала до углового расстояния, указанного в конец.

Углы указываются в градусах и отсчитываются от горизонтальной оси против часовой стрелки. Дуга рисуется против часовой стрелки, если конец положителен, и по часовой стрелке, если конец отрицателен. Поэтому, чтобы нарисовать дугу от 12-часового до 6-часового положения, начальный угол должен быть 90°и угол развертки 180°.

```
public void paint(Graphics g) {
g.drawArc(0, 40, 70, 70, 0, 75);
g. fillArc (0, 40, 70, 70, 0, 75); }
```

Работа с цветом

Работа с цветом поддерживается классом Color. В Color определено несколько цветовых констант (например color.black), специфицирующих ряд обычных цветов. Возможно также создание собственных цветов с применением одного из цветовых конструкторов. Обычно используются следующие его форматы:

```
Color (int red, int green, int blue)
Color (int rgbValue)
Color(float red, float green, float blue)
```

Пример

```
new Color(255, 100, 100); // светло-красный
```

По умолчанию графические объекты рисуются в текущем цвете переднего плана. Можно изменить этот цвет, вызывая метод setColor() класса Graphics:

```
void setColor(Color newColor)//
```
параметр newColor определяет новый цвет рисунка.

Вызывая метод getColor(), возможно получение текущего цвета:

```
Color getColor()
```

Порядок выполнения работы

1. Откройте среду разработки Eclipse.
2. Выполните задания.
3. Подготовьте отчёт.

Форма отчета о работе:

Лабораторная работа №…
Дата выполнения лабораторной работы №…
Тема лабораторной работы №…
Цель лабораторной работы №…
Оснащение лабораторной работы №…
Задание для выполнения лабораторной работы №…
Результаты выполнения лабораторной работы №…

1. *Условие задания для вашего варианта*
2. *Описание процесса реализации*
3. *Листинг кода программы*
4. *Скриншоты результата работы программы*

Ответы на контрольные вопросы к лабораторной работе №…
Вывод по лабораторной работе №…

Контрольные вопросы

1. Охарактеризуйте понятие GUI?
2. Какие пакеты используются для графического интерфейса приложений на Java?
3. Какие механизмы для управления содержит технология Swing?

Рекомендуемые источники

1. Г. Шилдт Java 8. Полное руководство. –Издательский дом «Вильямс», 2018.
2. Блинов, И.Н. Java 2: практ. рук. / И.Н. Блинов, В.С. Романчик. – Мн.: УниверсалПресс, 2005.

Приложение А

Варианты для задания

№ варианта (номер рабочего места)	Условие задания
1.	Создать классы Point и Line. Объявить массив из n объектов класса Point. Для объекта класса Line определить, какие из объектов Point лежат на одной стороне от прямой линии и какие − на другой. Реализовать ввод данных для объекта Line и случайное задание данных для объектов Point
2.	Создать классы Point и Line. Объявить массив из n объектов класса Point и определить в методе, какая из точек находится дальше всех от прямой линии, и пометить ее другим цветом
3.	Создать классы Point и Triangle. Объявить массив из n объектов класса Point. Написать функцию, определяющую какие из точек находятся внутри, а какие – снаружи треугольника. Нарисовать их разными цветами
4.	Создать классы Point и Rectangle. Объявить массив из n объектов класса Point. Написать функцию, определяющую какие из точек находятся внутри, а какие – снаружи прямоугольника. Нарисовать их разными цветами
5.	Определить класс Line для прямых линий, проходящих через точки A(x1,y1) и B(x2,y2). Создать массив объектов класса Line. Определить, используя функции, какие из прямых линий пересекаются, а какие − совпадают. Нарисовать все пересекающиеся прямые одним цветом, непересекающиеся − другим
6.	Создать класс Triangle. Определить, какие из m-введенных треугольников прямоугольные. Нарисовать их другим цветом
7.	Создать класс Triangle. Определить, какие из m-введенных треугольников имеют площадь, больше заданной. Прорисовать их другим цветом
8.	Создать классы Point и Circle. Объявить массив из n объектов класса Point. Для объекта класса Circle определить, какие из объектов Point лежат внутри окружности, а какие − вне. Реализовать ввод данных для объекта Circle и случайное задание данных для объектов Point